Hardware Trojan Detection at RTL Level

-CS203(Digital Logic Design) Project

Instructor :-Dr. Geeta

(CSE Department)

Team members

- Khush Varshney :2025CSB1206
- Jayanth Dahire :2025CSB1203(Team Lead)
- Jatin Singla :2025CSB1202
- Hanzala Zaid :2025CSB1197
- Gourav :2025CSB1193

Stage-1

•Understanding and Preparing the Plan of the Project

- **GOAL :** Learn UART basics and Trojan concepts

We had to choose two major aspects of our project:

1) System Design: Designing a system that was simple enough to implement using our existing knowledge and the concepts we learned during the project, yet complex enough to incorporate Trojans that would not be directly detectable.

2) Trojan Types and Detection Algorithm: Deciding which types of Trojans to inject into the system and developing an appropriate detection algorithm to identify them.

Understanding and Preparing the Plan of the Project

Upon exploring various systems in which we could implement trojans we ended up on a system which consisted of a transmitter and a receiver following UART protocol. We chose UART because:
Simple and understandable architecture
Contains sequential RTL logic
Easy to insert different Trojan triggers
Supports rare-trigger experiments
Observable outputs make functional testing straightforward

The trojans which were selected for injection:
Sequential Trojans: Trojan activated when certain sequence of inputs were given
FSM Trojans: Trojans whose trigger is determined by its current state and current input

Stage-2

Implementation of UART(Universal Asynchronous Receiver/Transmitter)

GOAL : Build a functional UART system

Activities:

1)Implemented transmitter, receiver , and top modules.

2)Defined FSM states: IDLE, START, DATA, STOP.

3)Added baud-rate counters and synchronizers.

4)Verified functionality using a loopback testbench.

Outcome: Established a working UART baseline (Golden Reference).

Stage-2

Implementation of UART(Universal Asynchronous Receiver/Transmitter)

1) **Difficulties faced:**

1)We had to understand the various specific syntax to troubleshoot the code.

2)Meta stability in receiver module: When a signal changes at the wrong time, a flip-flop may temporarily produce an unpredictable output instead of a clear 0 or 1.

3)We had to reset the whole transmitter even when it was not in sync with the clock.

4)We had to be careful about which edge of the clock cycle we were using to ensure synchronisation of parallel components and no race condition of series components.

Stage-2

Resources we used until this stage:

Sources:

1)Class Lectures PPT for Verilog syntax

2)NandLand UART Serial Port Module – Nandland Got ideas to implement the basic UART

3)LLM : Debugging

4)YouTube videos: For understanding syntax in Verilog

https://www.youtube.com/watch?v=YUB-OyGr1oA

Stage-3 (Construction of trojans and their detection)

- **GOAL** : Insert trojans and test detectability
- **Trojan types** :
- 1)Single byte trigger
- 2)Two-byte sequence trigger
- 3)Four-byte sequence trigger
- 4)Side effect trojan
- **Payloads** :
- 1)Payload A (Visible-Bit flip in output)
- 2)Payload B (Stealthy-No change in output. Internal change register change)

Stage-3 (Construction of trojans and their detection)

Testbench &Verification

Four Phases of Testing:

1)Exhaustive sweep(0-255bytes)

2)Pseudo-random stream

3)Near-Trojan sequences

4)Directed Trojan activation

Automatic: Self-checking golden model , timeout counters, per-section summaries

- Tools :
- 1) Verilog/ SystemVerilog
- 2)Icarus Verilog / Verilator
- 3)GTKWave / EPWave

Stage-3 (Construction of trojans and their detection)

Results & Detection Percentages

- **Exhaustive inputs:** 99.61% pass, Trojan detection ~0.39%
- **Random inputs:** 100% pass, Trojan detection 0%
- **Near-Trojan sequences:** 100% pass, Trojan detection 0%
- **Directed Trojan activation:** 100% detection
- **Key Insight:** Rare triggers evade ordinary testing, but directed tests expose them

- **Conclusion:**

•Findings:

- •Trigger rarity strongly affects detectability
- •Payload A visible → single byte trigger detected ,sequence trigger undetected
- •Payload B stealthy → undetected

•Final Goal Achieved: Single bit Trojan detected successfully as well as limitations and constraints of current solution determined

Stage-3 (Construction of trojans and their detection)

- **Limitations:**

•1) The four testbenches relied on ideal conditions and lacked real-world applicability.
•2) The testing approach required excessive computational resources, reducing efficiency.

- **Observation:**
- Our test bench was unable to detect any trojans except single-byte trigger trojans.
- There was no optimization of testing from our end.

- **Conclusion:**
- Upon discussion among ourselves and the TA's, we understood that EDA playground alone was not a strong enough tool and our solution was very brute-force.

Stage-3 (Construction of trojans and their detection)

Log | Share

```
============================================================
FINAL OVERALL TEST SUMMARY
============================================================
Total byte transactions : 548
Passed                  : 544
Failed                  : 4
RX timeouts             : 0
TX timeouts             : 0
RX error pulses         : 0
Failure details shown   : 4 (limit 40)
OVERALL RESULT: FAIL - inspect the per-section summaries and failure messages.
============================================================
testbench.sv:418: $finish called at 917540000 (1ps)
Done
```

```
------------------------------------------------------------
TEST TYPE 3: NEAR-TROJAN SEQUENCES
  Byte transactions : 29
  Passed            : 29
  Failed            : 0
  Pass percentage   : 100.00%
  RX timeouts       : 0
  TX timeouts       : 0
  RX error pulses   : 0
------------------------------------------------------------
TEST TYPE 4: DIRECTED TROJAN ACTIVATION
  Byte transactions : 7
  Passed            : 4
  Failed            : 3
  Pass percentage   : 57.14%
  RX timeouts       : 0
  TX timeouts       : 0
  RX error pulses   : 0
------------------------------------------------------------
```

```
============================================================
PER-TEST ANALYSIS SUMMARY (printed after all tests finish)
============================================================
TEST TYPE 1: ISOLATED EXHAUSTIVE INPUTS 0..255
  Byte transactions : 256
  Passed            : 255
  Failed            : 1
  Pass percentage   : 99.61%
  RX timeouts       : 0
  TX timeouts       : 0
  RX error pulses   : 0
------------------------------------------------------------
TEST TYPE 2: 256 PSEUDO-RANDOM BYTES
  Byte transactions : 256
  Passed            : 256
  Failed            : 0
  Pass percentage   : 100.00%
  RX timeouts       : 0
  TX timeouts       : 0
  RX error pulses   : 0
```

Stage-4 (Final implementation)

- **Solutions discussed**
- 1) Benign clouds-This algorithm suggests to keep a prefix and suffix sequences as reference to detect the trojan sequence.
- 2)De Bruijn-This algorithm produced a string of input that could repeat all possible sequences.
- 3) FSM trojans-We wanted to monitor every state change and determine the suspicious states.

- **Solution implemented**
- We switched from EDA playground to a Linux-based system so that we don't encounter any computational restrictions.
- We used Python to find the suspicious logic in the source code and generate test cases.
- We connected software automation with hardware simulation.

Stage-4 (Final implementation)

- **How it works?**

- The python script compares the clean RTL with the suspected RTL and finds out suspicious logic based on difference in codes ignoring whitespace and comments.
- The script assigns scores based on comparisons ,conditional-branches, state-variables, counters, data-paths, trigger-related names.
- Estimate trigger rarity using byte-sequence length, adjusted by detected FSM-state and variable constraints using formula listed at right side
- Filter and rank trigger candidates using the suspiciousness score and the configured rarity threshold of 1/256.
- Generate candidate input sequences and a System Verilog test configuration for selected trigger conditions.

$$P_{est} = \frac{(1/256)^{n_b}}{N_{\text{FSM}} + N_{\text{variables}} + 1}$$

Stage-4 (Final implementation)

- **Results:**
- We successfully detected trojans in the code using code comparisons as well as statistical analysis.
- We were able to optimize the search for trojans to a level which could be industry applicable.
- Multiple types of trojans could be detected using a general method without need for special cases.

- **Constraints:**
- This method requires complete access to both golden reference as well as the suspect code.
- The probability threshold is quite high for complex systems which have multiple logics embedded.
- The sequence score and probability formulas are generic and not absolute.
- Trojans hidden under broad conditions cannot be accessed.
- The trojan conditions must be quite descriptive.

Stage-4 (Final implementation)

- **MEDIUM (score 3)** `parameter [7:0] TROJAN_SINGLE_INPUT = 8'd94,` – trigger-related naming
- **MEDIUM (score 3)** `parameter [7:0] TROJAN_SINGLE_OUTPUT = 8'd95,` – trigger-related naming
- **MEDIUM (score 3)** `parameter [7:0] TROJAN_TWO_FIRST = 8'd36,` – trigger-related naming
- **MEDIUM (score 3)** `parameter [7:0] TROJAN_TWO_SECOND = 8'd37,` – trigger-related naming
- **MEDIUM (score 3)** `parameter [7:0] TROJAN_TWO_OUTPUT = 8'd38,` – trigger-related naming
- **MEDIUM (score 3)** `parameter [7:0] TROJAN_FOUR_FIRST = 8'd5,` – trigger-related naming
- **MEDIUM (score 3)** `parameter [7:0] TROJAN_FOUR_SECOND = 8'd27,` – trigger-related naming
- **MEDIUM (score 3)** `parameter [7:0] TROJAN_FOUR_THIRD = 8'd49,` – trigger-related naming
- **MEDIUM (score 3)** `parameter [7:0] TROJAN_FOUR_FOURTH = 8'd21,` – trigger-related naming
- **MEDIUM (score 3)** `parameter [7:0] TROJAN_FOUR_OUTPUT = 8'd22` – trigger-related naming
- **LOW (score 1)** `reg [2:0] state;` – state/history/counter reference
- **LOW (score 1)** `reg [7:0] prev1;` – state/history/counter reference
- **LOW (score 1)** `reg [7:0] prev2;` – state/history/counter reference
- **LOW (score 1)** `reg [7:0] prev3;` – state/history/counter reference
- **MEDIUM (score 2)** `state <= IDLE;` – comparison or assignment; state/history/counter reference
- **MEDIUM (score 2)** `prev1 <= 8'd0;` – comparison or assignment; state/history/counter reference
- **MEDIUM (score 2)** `prev2 <= 8'd0;` – comparison or assignment; state/history/counter reference
- **MEDIUM (score 2)** `prev3 <= 8'd0;` – comparison or assignment; state/history/counter reference
- **MEDIUM (score 3)** `if (rx_valid_in) begin` – conditional/control flow; possible observable data/control path
- **MEDIUM (score 2)** `case (state)` – conditional/control flow; state/history/counter reference
- **HIGH (score 7)** `if (rx_data_in == TROJAN_SINGLE_INPUT) begin` – conditional/control flow; comparison or assignment; possible observable data/control path; trigger-related naming
- **HIGH (score 6)** `rx_data_out <= TROJAN_SINGLE_OUTPUT;` – comparison or assignment; possible observable data/control path; trigger-related naming
- **MEDIUM (score 3)** `rx_data_out <= rx_data_in;` – comparison or assignment; possible observable data/control path
- **HIGH (score 7)** `if (rx_data_in == TROJAN_TWO_FIRST)` – conditional/control flow; comparison or assignment; possible observable data/control path; trigger-related naming
- **MEDIUM (score 2)** `state <= TWO_FIRST;` – comparison or assignment; state/history/counter reference
- **HIGH (score 7)** `else if (rx_data_in == TROJAN_FOUR_FIRST)` – conditional/control flow; comparison or assignment; possible observable data/control path; trigger-related naming
- **MEDIUM (score 2)** `state <= FOUR_FIRST;` – comparison or assignment; state/history/counter reference
- **HIGH (score 7)** `if (rx_data_in == TROJAN_TWO_SECOND)` – conditional/control flow; comparison or assignment; possible observable data/control path; trigger-related naming
- **HIGH (score 6)** `rx_data_out <= TROJAN_TWO_OUTPUT;` – comparison or assignment; possible observable data/control path; trigger-related naming
- **HIGH (score 7)** `if (rx_data_in == TROJAN_FOUR_FIRST)` – conditional/control flow; comparison or assignment; possible observable data/control path; trigger-related naming
- **HIGH (score 7)** `else if (rx_data_in == TROJAN_TWO_FIRST)` – conditional/control flow; comparison or assignment; possible observable data/control path; trigger-related naming
- **HIGH (score 7)** `if (rx_data_in == TROJAN_FOUR_SECOND)` – conditional/control flow; comparison or assignment; possible observable data/control path; trigger-related naming
- **MEDIUM (score 2)** `state <= FOUR_SECOND;` – comparison or assignment; state/history/counter reference
- **HIGH (score 7)** `if (rx_data_in == TROJAN_FOUR_THIRD)` – conditional/control flow; comparison or assignment; possible observable data/control path; trigger-related naming
- **MEDIUM (score 2)** `state <= FOUR_THIRD;` – comparison or assignment; state/history/counter reference
- **HIGH (score 7)** `if (rx_data_in == TROJAN_FOUR_FOURTH)` – conditional/control flow; comparison or assignment; possible observable data/control path; trigger-related naming
- **HIGH (score 6)** `rx_data_out <= TROJAN_FOUR_OUTPUT;` – comparison or assignment; possible observable data/control path; trigger-related naming

```
Rarity-guided UART Trojan detection results
Candidate input bytes: 10
Exact source-derived multi-byte sequences: 3
DETECTED length=1 sequence=[94] effects=DATA/VALID
Stage 1 tests: 256; new findings: 1
DETECTED length=2 sequence=[36, 37] effects=DATA/VALID
Stage 2 tests: 100; new findings: 1; broad enumeration=1
DETECTED length=3 sequence=[170, 187, 204] effects=SIDE_EFFECT
Stage 3 tests: 1000; new findings: 1; broad enumeration=1
DETECTED length=4 sequence=[5, 27, 49, 21] effects=DATA/VALID
Stage 4 tests: 10000; new findings: 1; broad enumeration=1

Total candidate tests: 11356
Unique findings: 4
Important: no finding does not prove the design is Trojan-free.
```

References:

- 1)*https://github.com/BarakBinyamin/Trojan-Detection*

B. Binyamin, “Trojan-Detection: Some Generic Probabilistic Methodologies to Identify Hardware Trojans in Arbitrary Hardware Designs,” GitHub repository. This project describes probabilistic hardware-Trojan detection using pseudorandom input/output comparisons which inspired implementation and methodology.

- 2)*https://ieeexplore.ieee.org/document/10942366*
- J. Cruz and J. Hamlet, “A Survey on the Design, Detection, and Prevention of Pre-Silicon Hardware Trojans,” IEEE Access, vol. 13, pp. 58509–58547, 2025. It surveys pre-silicon Trojan detection approaches, prevention methods, benchmarks, and limitations.
- 3)*https://ieeexplore.ieee.org/document/7086012*
- J. Zhang, F. Yuan, L. Wei, Y. Liu, and Q. Xu, “VeriTrust: Verification for Hardware Trust,” IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 34, no. 7, pp. 1148–1161, 2015. Useful for: identifying potential Trojan trigger inputs through verification and analysing suspicious hardware behaviour.
- 4)*https://dl.acm.org/doi/10.1145/2228360.2228378*
- S. Wei, K. Li, F. Koushanfar, and M. Potkonjak, “Hardware Trojan Horse Benchmark via Optimal Creation and Placement of Malicious Circuitry,” in Proceedings of the 49th Annual Design Automation Conference (DAC), 2012, pp. 90–95. It provides useful research context for evaluating Trojan-detection methods

Individual Roles:

1)Gourav :-Target system(UART) ,trojans ,debugging

2)Khush Varshney:-Target system(UART) , trojans ,final analysis

3)Hanzala Zaid:-Python integration ,trojans

4)Jatin Singla :-Python integration , rarity calculation

5)Jayanth Kumar Dahire :-Python integration ,test vector creation

GitHub Link : https://github.com/JayanthDahire/HTD

THANK YOU